

Miller–Rabin Primality Test

Part 1: The Problem

We need to answer:

“Is this number prime?”

For large numbers (100+ digits):

- Trial division × too slow
- Deterministic tests × expensive

👉 We use **Miller–Rabin**:

- Very fast ⚡
- Extremely accurate ✓
- Small probability of error

Part 2: Core Idea

If n is prime, then:

$$a^{n-1} \equiv 1 \pmod{n}$$

(This is from Fermat’s theorem)

Problem

Some composite numbers also satisfy this → **false positives**

👉 Miller–Rabin fixes this by checking *stronger conditions*

Part 3: Key Setup

We write:

$$n - 1 = 2^s \cdot d$$

Where:

- d is **odd**
- $s \geq 1$

Example

Let $n = 21$

$$20 = 2^2 \cdot 5$$

👉 $s = 2, d = 5$

Part 4: The Test (one round)

Pick a random number:

$$a \in [2, n - 2]$$

Step 1:

$$x = a^d \mod n$$

Step 2: Check

If:

- $x = 1$ OR
- $x = n - 1$

👉 Probably prime (pass this round)

Step 3: Repeat squaring

Compute:

$$x = x^2 \mod n$$

Repeat up to $s - 1$ times.

If you ever get:

$$x = n - 1$$

👉 pass

× Otherwise:

👉 Composite

Example (intuitive)

Test $n = 21$, pick $a = 2$

- $d = 5$

$$x = 2^5 \mod 21 = 32 \mod 21 = 11$$

Now square:

- $11^2 = 121 \mod 21 = 16$
- $16^2 = 256 \mod 21 = 4$

Never reached 20 $\rightarrow$ $\times$ composite

Part 5: Repeat for accuracy

Run test multiple times with different a :

- 1 round $\rightarrow$ maybe weak
- 5 rounds $\rightarrow$ very strong
- 10+ rounds $\rightarrow$ practically certain

👉 Error probability:

$$\leq \left(\frac{1}{4}\right)^k$$

Part 6: C++ Implementation

♦ Fast exponentiation

```
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;

    while (exp > 0) {
        if (exp & 1)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}
```

♦ Miller–Rabin Test

```
#include <iostream>
#include <cstdlib>
using namespace std;

long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;

    while (exp > 0) {
        if (exp & 1)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}

bool miller_rabin(long long n, int k = 5) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    // write n-1 = 2^s * d
    long long d = n - 1;
    int s = 0;
    while (d % 2 == 0) {
        d /= 2;
        s++;
    }

    for (int i = 0; i < k; i++) {
        long long a = 2 + rand() % (n - 3);
        long long x = mod_pow(a, d, n);

        if (x == 1 || x == n - 1)
            continue;

        bool composite = true;
        for (int j = 0; j < s - 1; j++) {
            x = (x * x) % n;
            if (x == n - 1) {
                composite = false;
                break;
            }
        }
    }

    if (composite)
        return false;
}
```

```
return true; // probably prime
```



Example usage

```
int main() {  
    cout << miller_rabin(17) << endl; // 1 (prime)  
    cout << miller_rabin(21) << endl; // 0 (composite)  
}
```

Exercises

◆ Understanding

1. Write $n - 1 = 2^s \cdot d$ for:
 - $n = 25$
 - $n = 91$

◆ Practice

2. Test manually:
 - $n = 15, a = 2$
 - $n = 17, a = 3$

Challenge

3. Modify code to:
 - test numbers up to 10^9
 - print all primes in a range

Important Notes

- This is **probabilistic**
- Not 100% certain, but:
 - error is extremely small
- Used in:
 - RSA key generation

- Blockchain systems
- Security libraries